Pitstop Oracle — Product Requirements

Version 3 | Improve the existing four-page application; validate demand before expanding

1. Executive summary

Pitstop Oracle is a fan-facing Formula 1 forecasting and exploration product. Its current Streamlit application has four main experiences: This Weekend, Race Explorer, Fantasy Lab, and Model Performance.

Direction: refine the existing application rather than rebuild it. Help fans understand the next Grand Prix, inspect a forecast, explore historical outcomes, and test scenarios.

The public README reports that the current ML model does not beat the pole-position baseline on its test set. Treat this as a transparency feature and a product-learning opportunity, not something to hide.

2. Current product baseline

Page	README-described behavior	Role
This Weekend	Selects the next GP and predicts, including before qualifying	Primary landing page
Race Explorer	Historical races; model prediction versus actual winner	Evidence and exploration
Fantasy Lab	Wet-race and grid-penalty what-if toggles	Scenario experience
Model Performance	Reports model performance against simple baselines	Trust and transparency

README-level pipeline: Jolpica API and Open-Meteo → ingest.py → raw files → features.py → driver_race.parquet → train.py → joblib models → Streamlit UI. The README describes dataCollectionService as a future/optional Postgres warehouse.

3. Users and jobs to be done

Primary audience: engaged F1 fans who follow race weekends and want context around contenders and uncertainty. Secondary audience: data-curious fans, creators, newsletter writers, and publishers.

• Before a race: understand likely contenders and forecast uncertainty.

• After qualifying: understand how the grid changes the outlook.

• Explore historical races and compare the model with actual outcomes and simple baselines.

• Test understandable hypothetical scenarios such as rain or grid penalties.

• Over a season: review the forecast track record.

4. Goals and non-goals

• Make the next-race forecast easy to understand and revisit.

• Show event, timestamp, forecast stage, data freshness, and model version.

• Compare fairly with pole sitter and other simple baselines.

• Explain scenario assumptions and limitations.

• Validate demand before building accounts, leagues, or payments.

Non-goals: no full rewrite; no claim that ML beats the pole baseline; no telemetry or paid-data dependency by default; no premature user-account or league system; no claim that Fantasy Lab is an official F1 fantasy optimizer.

5. Product requirements

This Weekend (P0): show Grand Prix, date, forecast stage, contender ranking, generated-at time, data-as-of time, model version, and data warnings. Display probabilities only if outputs are calibrated probabilities; otherwise call them scores or

rankings.

Race Explorer (P1): preserve historical browsing and model-versus-actual comparison. Distinguish historical replay from forecasts generated with information available at the time.

Scenario Lab (P1; current Fantasy Lab): retain wet-race and grid-penalty controls only if assumptions are implemented and testable. Show baseline versus scenario and label the result hypothetical. Consider renaming to Scenario Lab.

Model Performance (P0): show sample size, date range, target definition, split, and baselines. Separate pre-qualifying from post-qualifying evaluation and show model versions.

6. Success metrics and validation

Metric	Measurement
Activation	Visitors opening This Weekend and inspecting a contender
Repeat use	Returning visitors across consecutive GPs
Scenario engagement	Scenario usage and completion
Trust	User comprehension of forecast limitations
Prediction-game demand	Manual challenge submissions and repeat participation

Recruit 30–50 survey respondents across casual, regular, and data-oriented fans. Conduct 8–12 interviews. Test the current app through tasks, not only opinion questions. Run a manual prediction challenge over three GPs before building a social layer.

7. Delivery sequence and risks

• Phase 0 — Reliability: event selection, data freshness, forecast-stage labels, and error states.

• Phase 1 — Trust: baseline comparison, metric definitions, model versioning, and forecast history.

• Phase 2 — Experience: improve the existing four pages.

• Phase 3 — Demand: run a manual three-GP prediction challenge.

• Phase 4 — Expansion: add accounts, history, leagues, or publisher tools only when validated.

Open questions: are outputs calibrated probabilities or raw scores? Can historical data be reconstructed as of the forecast cutoff? Are pre- and post-qualifying evaluations separate? Do scenario toggles reflect a model or heuristic?

Scope limitation: this PRD is based on the public README and repository overview, not a complete source-code audit. Verify implementation details in ingest.py, features.py, train.py, predict.py, ui.py, and metrics.json.